PREINFORME DESAFÍO II

SALOME GÓMEZ MARTÍNEZ

TI. 1021926218

GRUPO 1

ANDRÉS DAVID GARCIA MORALES

CC 1062435555

GRUPO 5

INFORMÁTICA II

ANÍBAL JOSÉ GUERRA SOLER

AUGUSTO ENRIQUE SALAZAR JIMENEZ

UNIVERSIDAD DE ANTIOQUIA

MEDELLÍN

SEMESTRE 2026 - 2

09/10/2026

CONTEXTUALIZACIÓN

El desafío consiste en diseñar e implementar en **C++** la capa lógica de Ude@Book, una red social por consola para al menos 1000 usuarios que gestiona perfiles, relaciones de amistad, solicitudes, mensajes, álbumes y fotos. La solución debe basarse exclusivamente en Programación Orientada a Objetos sin herencia, con prohibición total de la STL y operando únicamente sobre estructuras de datos dinámicas de creación propia a partir de los datos cargados desde el almacenamiento permanente al iniciar. Para garantizar la eficiencia, el sistema se apoya en una lista genérica propia y una matriz de adyacencia optimizada en bits para representar la red de contactos. Finalmente, el programa debe ofrecer un menú interactivo para el inicio de sesión con visualización aleatoria de tres fotos de perfil de amigos, administración de amistades y solicitudes, visita al perfil de un amigo con sus álbumes en modo carrusel, sugerencias de contactos y un centro de mensajería, incluyendo además la medición del consumo de memoria e iteraciones al finalizar cada funcionalidad.

ARQUITECTURA DEL PROGRAMA

DESAFIO2

HEADERS

- Menu.h
- UdeABook.h
- SugerenciasAmigos.h
- Usuario.h
- Album.h
- Foto.h
- Mensaje.h
- Fecha.h
- Lista.h (plantilla completa)
- Medidor.h

SOURCES

- main.cpp

Menu.cpp
UdeABook.cpp
SugerenciasAmigos.cpp
Usuario.cpp
Album.cpp
Foto.cpp
Mensaje.cpp
Fecha.cpp
Medidor.cpp

El programa sigue una arquitectura modular en la que cada clase del diagrama tiene su pareja de archivos: el .h con la declaración y el .cpp con la implementación. La única excepción es Lista<T>, que por ser una plantilla se escribe completa en su .h. El archivo main.cpp solo crea el objeto principal y el menú y los pone a funcionar. Esta organización aporta las siguientes ventajas:

- **Separación de responsabilidades:** Menu se encarga únicamente de lo que ve el usuario en consola y delega cada opción en UdeABook, que es la clase administradora. Así la lógica del problema no queda mezclada con la interfaz.
- **Mantenibilidad:** cada entidad (Usuario, Album, Foto, Mensaje, Fecha) administra sus propios datos y operaciones. UdeABook solo coordina, lo que evita que se convierta en una clase gigante.
- **Encapsulamiento:** los encabezados exponen solo los métodos públicos necesarios (getters, setters y operaciones propias) y ocultan los detalles internos, como el bloque de bits de las amistades.

DIAGRAMA DE CLASES

El diseño tiene 10 clases y 13 relaciones. No se crearon clases para la solicitud de amistad, amistad, sesión, like y el manejo de archivos, porque no tienen datos propios: son un índice,

un bit, un contador o tareas de UdeABook. Todos los atributos son privados y un método es público solo si otra clase lo llama; por eso los ayudantes internos, como la fórmula del triángulo de bits o la ampliación de la lista, son privados.

- **CLASES Y RELACIONES**

UdeABook es la clase administradora: guarda la lista de todos los usuarios, el administrador de amistades y el índice del usuario con sesión abierta. Coordina las funcionalidades (cargar datos, iniciar sesión, elegir las tres fotos, enviar solicitudes y mensajes) y devuelve códigos de resultado para que la clase menu, que solo muestra pantallas y delega cada opción, informe al usuario. Usuario es la entidad central: tiene los datos del perfil, dos fechas y cuatro listas (álbumes, mensajes recibidos, mensajes enviados y solicitudes). Inserta los mensajes en su lugar según la fecha y ofrece una búsqueda binaria para el rango de fechas. Su identificador es su posición en la lista de UdeABook y en el triángulo de bits, y su constructor de copia es privado para no duplicar ni liberar dos veces su memoria.

Mensaje es un único objeto compartido: el receptor es su dueño y el emisor solo lo apunta, de modo que el estado leído es el mismo para ambos y el texto no se duplica. Album guarda su nombre, su privacidad y sus fotos además de que su método concentra la regla de quién puede verlo. Foto guarda la fecha, el tamaño, el nombre y los likes. De álbumes y fotos solo se guarda el nombre, y la ruta completa se arma al mostrarla. SugerenciasAmigos guarda las amistades en un bloque de bits y calcula los amigos en común y las sugerencias. Fecha guarda día, mes y año, sobrecarga. Medidor tiene todos sus miembros estáticos y un constructor privado, y su método contar tiene un valor por defecto.

Las relaciones son de dos tipos. Once composiciones (rombo lleno del lado del todo) indican que el todo crea y libera a sus partes: UdeABook contiene un administrador de

amistades, una lista y de cero a muchos usuarios; Usuario contiene álbumes y mensajes, cuatro listas y dos fechas; Album contiene sus fotos y una lista; y Foto y Mensaje contienen una fecha cada uno y dos asociaciones son colaboraciones sin propiedad: Menu usa a UdeABook y UdeABook mide con Medidor, ambas de 1 a 1. No se dibujan las relaciones de Usuario consigo mismo (amistad y solicitudes), porque se resuelven con índices, el triángulo de bits y la lista de solicitudes, ni la segunda relación entre Usuario y Mensaje, la del emisor que solo apunta.

Con este modelo, las funcionalidades se reparten de forma clara: en el ingreso, Menu pide las credenciales a UdeABook, que las valida con Usuario y elige las tres fotos usando SugerenciasAmigos; al visitar un perfil, UdeABook comprueba la amistad y Album decide si el álbum es visible; y en los mensajes, UdeABook crea el objeto y Usuario lo guarda ordenado.

GESTIÓN DE ARCHIVOS

El programa usa cuatro archivos de texto, uno por tipo de dato. Todo se carga a memoria al iniciar y, cuando algo cambia, el cambio también se escribe en el archivo correspondiente. Cada línea lleva escrito el id del usuario, de modo que el cargador no depende del orden del archivo.

- **usuarios.txt** tiene una línea por usuario con id, nombre de usuario, contraseña, correo, ciudad, país, fecha de nacimiento y fecha de inscripción; no cambia durante la ejecución.
- **amistades.txt** tiene una línea por usuario con la lista de sus amigos y la de las solicitudes que ha recibido, ambas separadas por comas; cada amistad aparece dos veces, una en la línea de cada amigo.

- **mensajes.txt** tiene una línea por mensaje con emisor, receptor, fecha, estado leído (0 o 1) y texto, que va al final porque puede contener punto y coma.
- **albumes.txt** tiene la línea de cada álbum (usuario, nombre, privacidad y cantidad de fotos) seguida de una línea por cada foto (nombre, fecha, tamaño y likes). Solo se guardan los nombres; las rutas completas se arman al mostrarlas, y la cantidad de fotos permite crear cada lista con su tamaño exacto.

La carga sigue un orden: primero los usuarios, que indican cuántos hay y permiten reservar el triángulo de bits; luego las amistades y solicitudes; después los mensajes, que se insertan por fecha en el receptor y en el emisor; y por último los álbumes con sus fotos.Para actualizar, lo nuevo se agrega al final del archivo (enviar un mensaje no requiere leer nada). Lo existente (aceptar una solicitud, marcar mensajes como leídos, dar un like) se cambia recorriendo el archivo línea por línea, copiándolo a un archivo temporal con la línea modificada y devolviendo luego el contenido al original.

Se eligió una línea por usuario en las amistades porque el costo de recorrer el archivo depende de las líneas leídas: son unas 1.000 frente a unas 25.000 con una línea por amistad. Los mensajes llevan una línea cada uno porque enviar es la acción más repetida y solo agrega una línea, y porque guardar todos los de un usuario en una línea los duplicaría y la haría muy larga.

CONCLUSIÓN

En conclusión, se plantea Ude@Book mediante un modelo de 10 clases sin herencia, con atributos privados y una clase controladora que coordina el sistema donde se define la propiedad clara de los objetos como los mensajes compartidos optimizando la arquitectura y previniendo errores de memoria y finalmente , en términos de eficiencia, el uso de una lista genérica dinámica y una matriz triangular de bits para las relaciones permite gestionar

más de 1,000 usuarios con un consumo de memoria mínimo y consultas rápidas de amigos en común o sugerencias además de que los mensajes ordenados por fecha optimizan las búsquedas por rango mediante búsqueda binaria, mientras que la selección de fotos de perfil aplica un muestreo ponderado sin repeticiones.